

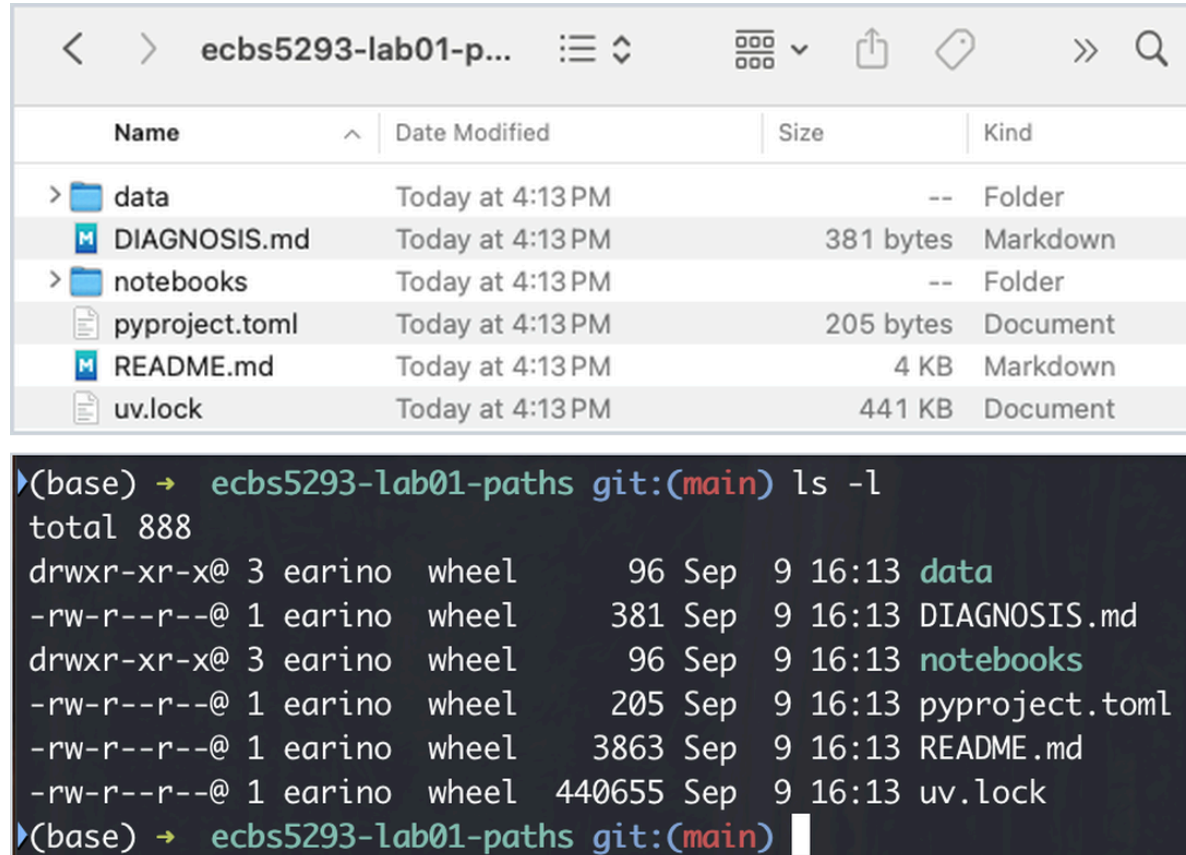
Computing for Analytical Work

Session 1 — Where am I?

Block 2 — The terminal as a way to inspect reality

This block in one sentence

The terminal is not a different computer. It is a precise way to talk to the same computer.



The image shows two side-by-side views of the same directory, `ecbs5293-lab01-paths`. The top view is a Finder window showing a tidy list of files and folders. The bottom view is a terminal window showing the same list plus exact sizes, times, owners, and permissions.

Name	Date Modified	Size	Kind
> data	Today at 4:13 PM	--	Folder
DIAGNOSIS.md	Today at 4:13 PM	381 bytes	Markdown
> notebooks	Today at 4:13 PM	--	Folder
pyproject.toml	Today at 4:13 PM	205 bytes	Document
README.md	Today at 4:13 PM	4 KB	Markdown
uv.lock	Today at 4:13 PM	441 KB	Document

```
(base) → ecbs5293-lab01-paths git:(main) ls -l
total 888
drwxr-xr-x@ 3 earino wheel   96 Sep  9 16:13 data
-rw-r--r--@ 1 earino wheel  381 Sep  9 16:13 DIAGNOSIS.md
drwxr-xr-x@ 3 earino wheel   96 Sep  9 16:13 notebooks
-rw-r--r--@ 1 earino wheel  205 Sep  9 16:13 pyproject.toml
-rw-r--r--@ 1 earino wheel 3863 Sep  9 16:13 README.md
-rw-r--r--@ 1 earino wheel 440655 Sep  9 16:13 uv.lock
(base) → ecbs5293-lab01-paths git:(main)
```

Same folder, same six entries, two views. Finder shows a tidy list. The terminal shows the same list plus exact sizes, times, owners, and permissions.

Same on Windows

Name	Date modified	Type	Size
.env	10/09/2026 00:09	File folder	
data	10/09/2026 00:08	File folder	
notebooks	10/09/2026 00:08	File folder	
.gitignore	10/09/2026 00:08	Git Ignore Source ...	1 KB
.python-version	10/09/2026 00:08	PYTHON-VERSIO...	1 KB
DIAGNOSIS.md	10/09/2026 00:08	Markdown Source...	1 KB
pyproject.toml	10/09/2026 00:08	Toml Source File	1 KB
README.md	10/09/2026 00:08	Markdown Source...	5 KB
uv.lock	10/09/2026 00:08	LOCK File	431 KB

```
Nahian@hp MINGW64 ~/ecbs5293-setup-check (main)
$ cd ~/Desktop/lab01
ls -l
total 442
-rw-r--r-- 1 Nahian 197121    381 Sep 10 00:08 DIAGNOSIS.md
-rw-r--r-- 1 Nahian 197121   4263 Sep 10 00:08 README.md
drwxr-xr-x 1 Nahian 197121     0 Sep 10 00:08 data/
drwxr-xr-x 1 Nahian 197121     0 Sep 10 00:08 notebooks/
-rw-r--r-- 1 Nahian 197121    205 Sep 10 00:08 pyproject.toml
-rw-r--r-- 1 Nahian 197121 440655 Sep 10 00:08 uv.lock

Nahian@hp MINGW64 ~/Desktop/lab01 (main)
$ |
```

File Explorer and Git Bash on the same folder. Explorer here is set to show hidden files, so it lists three more:

`.env`, `.gitignore`, `.python-version`. `ls` hides dot-files unless you ask: `ls -la`.

Agenda

1. Two-minute recap: `pwd` , `ls` , `cd`
2. Shell, prompt, command, output — and the Windows rule
3. Making and moving things: `mkdir` , `cp` , `mv`
4. Looking inside files without Excel: `head` , `tail` , `wc`
5. Running a script with `uv run python ...`
6. Recording the commands that worked
7. Git thread: `git diff` . AI thread: ask before you run.
8. Stretch, Lab 2, Homework 1

Two-minute recap: `pwd`, `ls`, `cd`

```
pwd          # where am I?  
ls           # what is here?  
cd notebooks # go there; every relative path now starts from there  
cd ..        # up one level
```

You used all four in Lab 1. If any of them still feels foreign, tell a TA during the lab. We are not re-teaching them; we are building on them.

What a shell is

- The **terminal** is the window. The **shell** is the program inside it that reads what you type.
- The shell shows a **prompt**, waits, you type a **command**, press Enter, it runs it, prints the **output**, and shows the prompt again.
- That loop is the whole thing. Prompt, command, output, repeat.

```
anna@laptop project $ ls data/raw  
sales_2024.csv  
anna@laptop project $
```

Anatomy of a command

ls **-l** **data/raw**

1 2 3

- 1 **command** the program to run
- 2 **option (flag)** changes how it behaves: here, a long listing
- 3 **argument** what to act on: a path, resolved from the cwd

```
anna@laptop project $ ls -l data/raw
total 96
-rw-r--r--  1 anna  staff  48211 Sep  3 10:42 sales_2024.csv
```

size in bytes last modified

evidence, before you
open anything

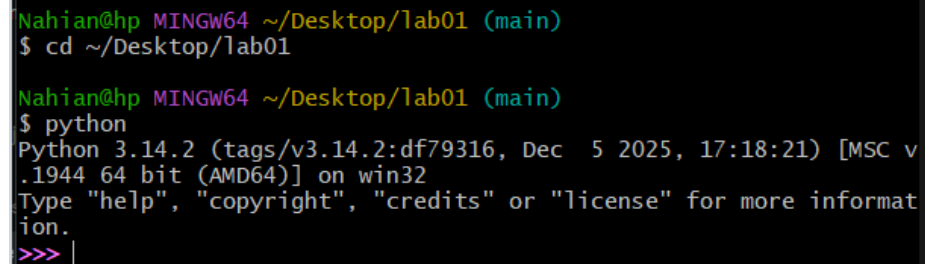
Spaces separate the pieces. A path with a space in it needs quotes: `"My Data/raw"`. Better: no spaces in project names.

Supported shells and the Windows rule

- **macOS / Linux:** the system Terminal. **Windows:** **Git Bash**, installed with Git for Windows. PowerShell and `cmd.exe` are at your own risk; TAs will not debug them in lab. Every command in this course works identically in both.

Windows rule: never type bare `python` in Git Bash. On some setups it hangs with no error; on others it drops you into Python's `>>>` prompt. Neither is where you want to be, and you cannot tell in advance which you have, so always pass an argument:

```
python --version
python -c "import sys; print(sys.executable)"
```



```
Nahian@hp MINGW64 ~/Desktop/lab01 (main)
$ cd ~/Desktop/lab01

Nahian@hp MINGW64 ~/Desktop/lab01 (main)
$ python
Python 3.14.2 (tags/v3.14.2:df79316, Dec  5 2025, 17:18:21) [MSC v
.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more informat
ion.
>>> |
```

If it *does* run and you see `>>>`, you are inside Python, not the shell. `exit()` gets you back.

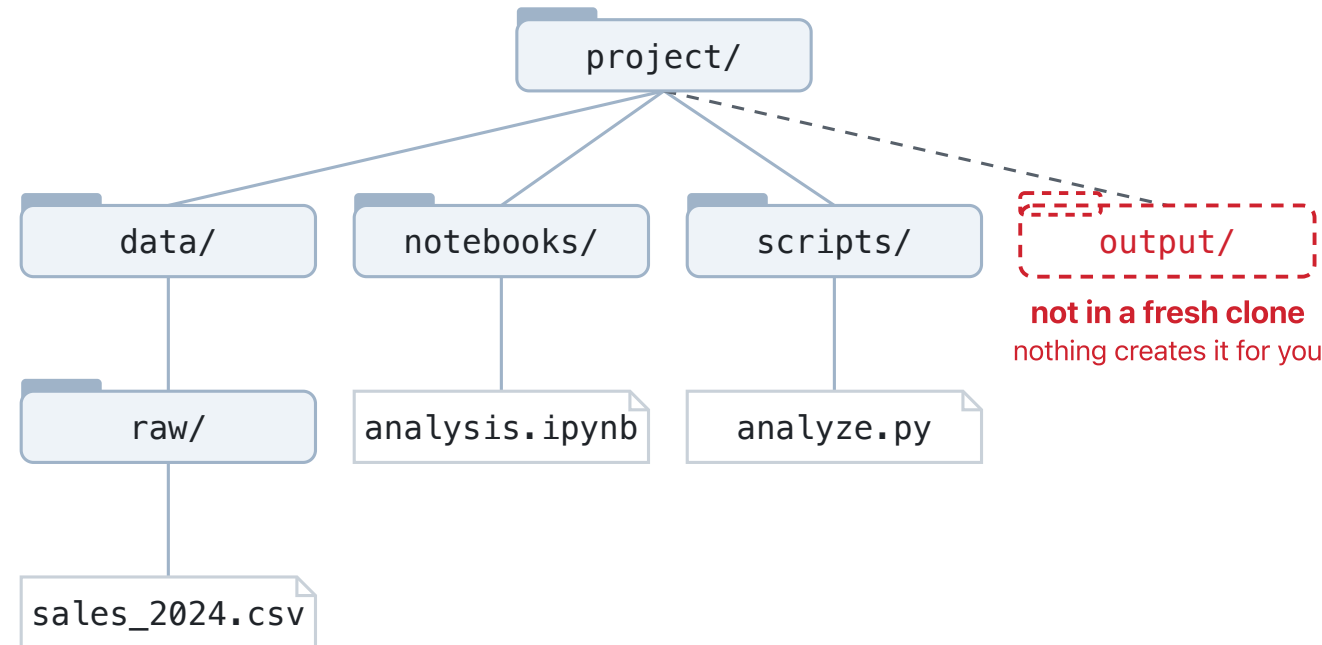
Commands that change things

`mkdir`, `cp`, `mv`

`mkdir` — make a folder

```
mkdir output
mkdir -p data/processed
ls
```

- Relative to the cwd, like every other path. — `p` makes the parents too and stays quiet if the folder exists — safe to run twice, which is why run instructions use it. Then `ls` to confirm.
- Programs do **not** create folders for you when they write files. A script that writes `output/summary.csv` into a missing `output/` fails. Remember this in the lab.



cp — copy

```
cp data/raw/sales_2024.csv data/raw/sales_backup.csv # file → new file  
cp -r data data_backup # folder → folder, recursively
```

- Source first, destination second. Both relative to the cwd.
- Copying makes **two** files. Now there are two truths. Before you copy data, ask whether you actually want that.

mv — move, which is also rename

```
mv notebooks/sales_2024.csv data/raw/sales_2024.csv # move to the right place
mv analyze.py analyse.py                          # same folder: that is a rename
```

- One file remains. That is the honest fix for a misplaced file.
- **mv** will **silently overwrite** a file at the destination. Look first: `ls data/raw`.

Predict: `mv` from the wrong place

You are standing in `project/notebooks/`. You type:

```
mv notebooks/sales_2024.csv data/raw/sales_2024.csv
```

What happens? Decide before the next slide. Hands: moves, or fails?

mv from the wrong place — what happened

```
mv: notebooks/sales_2024.csv: No such file or directory
```

- Both arguments resolve from the cwd. The shell looked for `notebooks/notebooks/sales_2024.csv`.
- Same rule as Block 1, now with a command that **changes** things.
- Had it succeeded from the wrong place, it would have said nothing. Look, then act.

Commands that only look

`head`, `tail`, `wc`

head — the first lines

```
head data/raw/sales_2024.csv      # first 10 lines
head -n 3 data/raw/sales_2024.csv # first 3
```

```
anna@laptop project $ head -n 3 data/raw/sales_2024.csv
```

1 date,region,product,units,unit_price

3 2024-03-13, North, Doohickey, 5, 4.75

2024-06-19,North,Widget,33,12.5

1 **the header row**

five names your code must match exactly

2 **the delimiter**

a comma

3 **the date format**

ISO: year-month-day

4 **the decimal mark**

a point, not a comma

You now know the delimiter, the header, the date format, and the column names. Before writing a line of code.

tail — the last lines

```
tail data/raw/sales_2024.csv      # last 10 lines
tail -n 2 data/raw/sales_2024.csv # last 2
```

- Files often go wrong at the end: a truncated download, a trailing "Total" row, a blank line, a footer someone typed by hand.
- **head** and **tail** sample the two **ends**. Together they tell you whether the ends agree. The middle stays unseen: **wc -l** is the next check, and a full pass comes in Session 3.

wc — how big is it?

```
wc -l data/raw/sales_2023.csv      # count lines  
wc -l data/raw/*.csv              # all CSVs at once
```

```
1201 data/raw/sales_2023.csv
```

1,201 lines. If the header is one line, every record is one line, and nothing trails at the end: 1,200 rows. Say those assumptions when you subtract; a quoted field with a line break in it breaks the arithmetic. When your dataframe says 900, you have a number to argue with.

Why not just open it in Excel?

English (Austria): every row in one cell

POSSIBLE DATA LOSS Some features might be lost if you save this workbook in the current format.

B11 :

	A	B	C	D	E	F	G
1	id,sku,ratio,date,revenue,note						
2	00042,1E3,3/4,2024-03-04,"1,440.00",checked						
3	00043,2E5,1/2,2024-03-05,"840.50",ok						
4	00044,7B2,2/3,2024-03-06,"12,000.00",resend						
5							
6							

English (United States): values rewritten

	A	B	C	D	E	F	G
1	id	sku	ratio	date	revenue	note	
2	42	1.00E+03	4-Mar	3/4/2024	1,440.00	checked	
3	43	2.00E+05	2-Jan	3/5/2024	840.5	ok	
4	44	7B2	3-Feb	3/6/2024	12,000.00	resend	

- Excel does not show you the file. It shows you its **interpretation**, and the interpretation depends on the laptop: one CSV, two regional formats, two different wrong answers.
- Then you click Save, and the file on disk is different from the one you were given.
- `head` shows the bytes. It cannot change them.

Rule for the course: inspect with the terminal; analyse in Python; open in Excel only when you mean to edit.

Predict: a script run from `scripts/`

Two launches. Inside the script, both times, the code opens `data/raw/sales_2024.csv`.

```
cd scripts && uv run python analyze.py           # launch A
cd .. && uv run python scripts/analyze.py         # launch B, back at the project root
```

For each launch, resolve **two** paths from the shell's cwd, and decide whether each exists in the project tree from Block 1 (`data/raw/sales_2024.csv` , `notebooks/analysis.ipynb` , `scripts/analyze.py`):

1. The **script** path — the thing you typed after `python` .
2. The **data** path — the thing the script opens.

Decide, then the next slide.

Running a script

```
cd ~/where/you/cloned/it          # your project folder (the root); not a literal path
uv run python scripts/analyze.py  # launch B: the script path and every path inside it resolve from here
```

- **Launch A**, from `scripts/`: the script path `analyze.py` resolves to `scripts/analyze.py`, exists. The data path resolves to `scripts/data/raw/sales_2024.csv`, does not.
- **Launch B**, from the root: `scripts/analyze.py` exists, and the data path resolves to `data/raw/sales_2024.csv`, exists. The cwd changed, so both paths changed meaning, and the command had to change to match.
- The script's cwd is **where you ran it from**, not where the `.py` file lives. Same rule as the notebook kernel in Block 1. Output goes wherever the script's relative paths point — from the cwd.

Sidebar: you have been typing `uv run` since setup

- The pre-course checklist had you run `uv sync` and `uv run python -c "import pandas; ..."`.
- Lab 1 had you run `uv sync` and pick the `.venv` kernel. Lab 2 is `uv run python scripts/summarize.py`.
- For now it is a **recipe**: type it exactly, from the project root.
- Session 2, Block 3 opens by explaining what it has been doing all along. Until then, do not substitute bare `python`, and do not ask why. Just insist on it.

Recording the commands that worked

Keep a plain text file — `COMMANDS.md`, `notes.txt`, whatever — and paste in the exact lines that worked, in order, with the folder you were in.

```
# from the project root
uv sync
mkdir -p output
uv run python scripts/summarize.py
ls output
```

Not the ones you tried. The ones that **worked**. That file becomes your README's run instructions — and *those* are a homework deliverable.

Git thread — `git diff`: what did I change?

```
git status      # which files changed?  
git diff        # what changed inside them?
```

```
--- a/scripts/analyze.py  
+++ b/scripts/analyze.py  
-OUT = Path("../output/summary.csv")  
+OUT = Path("output/summary.csv")
```

Lines starting with `-` are what was there. Lines with `+` are what you wrote. Everything else is context.

`git diff` covers **tracked** files only. A file you created is not in it. A file you *moved* shows as a deletion at the old place and nothing at the new one; `git status` lists the new location as *untracked*.

AI thread — ask before you run

I am in Git Bash on Windows, in the root of a Python project.
Before I run it, what does this command do, step by step?

```
rm -rf output/
```

Is there a version that only deletes CSV files, and asks before each one?

- **Explain before executing.** Especially anything with `rm`, `mv`, `-r`, `-f`, or `*`.
- **Ask for a safer version** of a destructive command. `rm -i`, a narrower pattern, a dry run.
- Then read the explanation, and decide.

AI thread — you are responsible for what runs on your machine

- The assistant did not press Enter. You did.
- "AI told me to" is not a diagnosis, not an excuse, and not something you can explain to the person next to you.
- AI can help interpret commands and output. **You are responsible for what runs on your machine.**
- Good uses today: explain this command, explain this output, give me a safer version. Bad use: paste a block you cannot explain and run it.

5-minute stand-and-stretch

Stand up. Leave the laptop. Water, window, hallway.

Back in five minutes, then 45 minutes of lab. Homework 1 is on the last slide; it will still be there when you return.

Lab 2 — Run the project without clicking around

`scripts/summarize.py` reads a CSV with the standard library only — no pandas; this lab is about paths and the shell. First inspect the input with `head` and `wc -l`, as the README says. Then follow the README's *Reproduce the failure* block exactly and read what happens. Then the loop: observe → locate → hypothesize → inspect → change → verify. There is more than one thing wrong, and the first fix is not the last. Record the exact commands that worked. `git diff` to see your change. One diagnosis note per thing you fixed. Explain it to a neighbour at minute 33. Checkpoint.

Terminal, in your course folder — type this exactly, then the README's *Reproduce the failure* block:

```
git clone https://github.com/earino/ecbs5293-lab02-terminal.git
cd ecbs5293-lab02-terminal
uv sync
```

Stretch: make the script robust to the second thing you found, so the next person never meets it. And: why would this also run with bare `python scripts/summarize.py` and no `uv sync`? Hold that for Session 2.

The last ten minutes — explain it to a neighbour

Speaker, about a minute, pointing at the screen, not reading the note: symptom → cause → evidence → change → verification.

Listener asks:

1. Show me the evidence.
2. Why did it fail — not just where?
3. Your fix is in. Now `cd scripts` and run `uv run python summarize.py`. What happens, and why?

Swap. Unsure, or you disagree? **Hand up**. Then the answer, for everyone. Before you leave: `DIAGNOSIS.md` to the Lab 2 checkpoint on Moodle.

Homework 1 assigned — Files, paths, terminal, and run instructions

A colleague's project that does not run as documented. Make it run, prove the report came from the supplied data, and document how. Expected time: **5–6 hours**. Due the **Friday after this session, 23:59** — slot on Moodle.

Deliverables, on Moodle:

1. The fixed repo with corrected paths. The grader regenerates the output from your run instructions — so show in your note that it did.
2. Run instructions in the README: the exact commands that worked, in order, from the project folder
3. A diagnosis note per failure: symptom, cause, evidence, change, verification — with the sanity counts (rows in, rows loaded, products out) **and the reconciliation**: revenue totalled from the input equals the report's total, to the cent
4. A **60–90 second video walkthrough**: screen recording, run the fixed code, then for every failure you fixed: what broke, why, what you changed, how you verified. No script reading.
5. The zip, made by one command in the README: `uv run python make_submission.py`. No Git needed yet — Git today is `status` and `diff`, and the diff belongs in your note.